Cubo 3D con Python y OpenGL

https://gemini.google.com/app/4c3399cde0f322b5

User prompt: Gráfica de cubo con OpenGL y Python

Response: Para dibujar y animar un cubo 3D interactivo con Python y OpenGL, la combinación más sencilla y robusta es usar **PyOpenGL** junto con **Pygame** para gestionar la ventana y la captura de teclado/ratón.

Requisitos previos

Primero instala las bibliotecas necesarias desde la terminal:

```
pip install PyOpenGL PyOpenGL_accelerate pygame
```

Código completo

Guarda el siguiente código en un archivo `.py` y ejecútalo. Podrás **rotar el cubo** arrastrando con el ratón o usando las teclas de flecha:

```
import pygame
from pygame.locals import *
from OpenGL.GL import *
from OpenGL.GLU import *

# 1. Definición de la geometría del cubo
vertices = (
    ( 1, -1, -1),
    ( 1,  1, -1),
    (-1,  1, -1),
    (-1, -1, -1),
    ( 1, -1,  1),
    ( 1,  1,  1),
    (-1, -1,  1),
    (-1,  1,  1)
)

# Definición de las 6 caras uniendo sus vértices
faces = (
    (0, 1, 2, 3),
    (3, 2, 7, 6),
    (6, 7, 5, 4),
    (4, 5, 1, 0),
    (1, 5, 7, 2),
    (4, 0, 3, 6)
)

# Un color único (RGB) para cada una de las 6 caras
colors = (
    (1, 0, 0),  # Rojo
    (0, 1, 0),  # Verde
    (0, 0, 1),  # Azul
    (1, 1, 0),  # Amarillo
    (1, 0, 1),  # Magenta
    (0, 1, 1),  # Cian
)

# Definición de las aristas del cubo (líneas negras)
edges = (
    (0,1), (1,2), (2,3), (3,0),
    (4,5), (5,7), (7,6), (6,4),
    (0,4), (1,5), (2,7), (3,6)
)

def draw_cube():
    # Dibujar las caras sólidas
    glBegin(GL_QUADS)
    for i, face in enumerate(faces):
        glColor3fv(colors[i])
        for vertex in face:
            glVertex3fv(vertices[vertex])
    glEnd()

    # Dibujar las aristas del cubo para dar contraste
    glColor3f(0, 0, 0)
    glLineWidth(2)
```

```
        glBegin(GL_LINES)
        for edge in edges:
            for vertex in edge:
                glVertex3fv(vertices[vertex])
        glEnd()

    def main():
        pygame.init()
        display = (800, 600)
        pygame.display.set_mode(display, DOUBLEBUF | OPENGL)
        pygame.display.set_caption("Cubo 3D - PyOpenGL")

        # Configuración de perspectiva 3D y cámara
        gluPerspective(45, (display[0] / display[1]), 0.1, 50.0)
        glTranslatef(0.0, 0.0, -5.0)  # Alejar el cubo en el eje Z

        # Habilitar prueba de profundidad para renderizado 3D correcto
        glEnable(GL_DEPTH_TEST)

        rot_x, rot_y = 0, 0
        mouse_down = False
        last_mouse = (0, 0)
        clock = pygame.time.Clock()

        running = True
        while running:
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    running = False
                elif event.type == KEYDOWN and event.key == K_ESCAPE:
                    running = False

                # Control por ratón
                elif event.type == MOUSEBUTTONDOWN and event.button == 1:
                    mouse_down = True
                    last_mouse = event.pos
                elif event.type == MOUSEBUTTONUP and event.button == 1:
                    mouse_down = False
                elif event.type == MOUSEMOTION and mouse_down:
                    dx = event.pos[0] - last_mouse[0]
                    dy = event.pos[1] - last_mouse[1]
                    rot_x += dy * 0.5
                    rot_y += dx * 0.5
                    last_mouse = event.pos

            # Control continuo por teclado (Flechas)
            keys = pygame.key.get_pressed()
            if keys[K_LEFT]:  rot_y -= 2
            if keys[K_RIGHT]: rot_y += 2
            if keys[K_UP]:    rot_x -= 2
            if keys[K_DOWN]:  rot_x += 2

            # Limpiar pantalla y búfer de profundidad
            glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

            # Aplicar la rotación acumulada
            glPushMatrix()
            glRotatef(rot_x, 1, 0, 0)
            glRotatef(rot_y, 0, 1, 0)

            # Dibujar el cubo
            draw_cube()

            glPopMatrix()

            pygame.display.flip()
            clock.tick(60)

        pygame.quit()

    if __name__ == "__main__":
        main()
```

Explicación clave

- **Vértices y Caras:** El cubo se compone de 8 puntos 3D. Las 6 caras se forman mediante cuadriláteros (`GL_QUADS`), conectando 4 vértices cada una.
- **Prueba de Profundidad (`GL_DEPTH_TEST`):** Esencial en 3D para asegurarte de que las caras frontales oculten a las traseras correctamente.
- **Transformaciones (`glRotatef` / `glTranslatef`):** Alejan el cubo del espectador e imprimen la rotación en los ejes X e Y antes de ejecutar las instrucciones de dibujo.

User prompt: ¿Cómo puedo añadir iluminación básica (luz ambiental y difusa) al cubo de PyOpenGL?

Response: Para añadir iluminación al cubo con PyOpenGL (usando el modelo de sombreado Phong/Lambert en OpenGL clásico), necesitas configurar tres elementos clave: **activar la iluminación**, **definir la posición/propiedades de la fuente de luz** y **asignar un vector normal a cada cara** del cubo para que OpenGL sepa cómo rebota la luz en cada superficie.

Código modificado con iluminación

Sustituye tu código anterior por este. Incluye iluminación ambiental, difusa y la asignación de normales por cara:

```
import pygame
from pygame.locals import *
from OpenGL.GL import *
from OpenGL.GLU import *

# 1. Geometría del cubo
vertices = (
    ( 1, -1, -1), ( 1,  1, -1), (-1,  1, -1), (-1, -1, -1),
    ( 1, -1,  1), ( 1,  1,  1), (-1, -1,  1), (-1,  1,  1)
)

faces = (
    (0, 1, 2, 3), # Trasera
    (3, 2, 7, 6), # Izquierda
    (6, 7, 5, 4), # Frontal
    (4, 5, 1, 0), # Derecha
    (1, 5, 7, 2), # Superior
    (4, 0, 3, 6)  # Inferior
)

# 2. Vectores Normales perpendiculares a cada cara (indican hacia dónde "mira" la superficie)
normals = (
    ( 0,  0, -1), # Trasera
    (-1,  0,  0), # Izquierda
    ( 0,  0,  1), # Frontal
    ( 1,  0,  0), # Derecha
    ( 0,  1,  0), # Superior
    ( 0, -1,  0)  # Inferior
)

def setup_lighting():
    glEnable(GL_LIGHTING)
    glEnable(GL_LIGHT0)
    glEnable(GL_COLOR_MATERIAL) # Permite usar glColor() para definir el color del material

    # Propiedades de la luz GL_LIGHT0
    ambient_light  = [0.2, 0.2, 0.2, 1.0]  # Luz ambiental tenue
    diffuse_light  = [0.8, 0.8, 0.8, 1.0]  # Luz difusa blanca brillante
    light_position = [3.0, 4.0, 3.0, 1.0]  # Posición (X, Y, Z, W) en el espacio 3D

    glLightfv(GL_LIGHT0, GL_AMBIENT, ambient_light)
    glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuse_light)
    glLightfv(GL_LIGHT0, GL_POSITION, light_position)

    # Configuración de propiedades del material para reaccionar a la luz
    glColorMaterial(GL_FRONT, GL_AMBIENT_AND_DIFFUSE)

def draw_cube():
    glBegin(GL_QUADS)
    for i, face in enumerate(faces):
        glNormal3fv(normals[i]) # Definir la normal antes de dibujar los vértices de la cara
        glColor3f(0.2, 0.6, 0.9) # Color azul uniforme para apreciar bien el sombreado
        for vertex in face:
            glVertex3fv(vertices[vertex])
    glEnd()
```

```
def main():
    pygame.init()
    display = (800, 600)
    pygame.display.set_mode(display, DOUBLEBUF | OPENGL)
    pygame.display.set_caption("Cubo 3D con Iluminación - PyOpenGL")

    gluPerspective(45, (display[0] / display[1]), 0.1, 50.0)
    glTranslatef(0.0, 0.0, -5.0)

    glEnable(GL_DEPTH_TEST)
    setup_lighting() # Activar y configurar iluminación

    rot_x, rot_y = 0, 0
    mouse_down = False
    last_mouse = (0, 0)
    clock = pygame.time.Clock()

    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT or (event.type == KEYDOWN and event.key == K_ESCAPE):
                running = False
            elif event.type == MOUSEBUTTONDOWN and event.button == 1:
                mouse_down = True
                last_mouse = event.pos
            elif event.type == MOUSEBUTTONUP and event.button == 1:
                mouse_down = False
            elif event.type == MOUSEMOTION and mouse_down:
                dx = event.pos[0] - last_mouse[0]
                dy = event.pos[1] - last_mouse[1]
                rot_x += dy * 0.5
                rot_y += dx * 0.5
                last_mouse = event.pos

        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

        glPushMatrix()
        glRotatef(rot_x, 1, 0, 0)
        glRotatef(rot_y, 0, 1, 0)

        draw_cube()

        glPopMatrix()

        pygame.display.flip()
        clock.tick(60)

    pygame.quit()

if __name__ == "__main__":
    main()
```

Conceptos clave agregados

- **`glNormal3fv(normals[i])`:** La física del sombreado difuso calcula la iluminación según el ángulo entre el rayo de luz y la perpendicular a la cara (la **normal**). Sin este comando, OpenGL no sabría hacia dónde "mira" cada plano.
- **Luz Ambiental (`GL_AMBIENT`):** Define la luz base uniforme presente en toda la escena. Evita que las caras que no reciben luz directa queden completamente en negro.
- **Luz Difusa (`GL_DIFFUSE`):** Define el brillo principal de la luz en función del ángulo en que impacta sobre cada cara.
- **`GL_COLOR_MATERIAL`:** Le indica a OpenGL que tome los colores establecidos mediante `glColor3f()` como las propiedades reflectivas del material del objeto.

User prompt: ¿Cómo añado componente especular (brillo de espejo/plástico) a la iluminación del cubo en PyOpenGL?

Response: Para añadir el **brillo especular** (el reflejo brillante tipo plástico o metal brillante), necesitas habilitar y configurar la luz especular (`GL_SPECULAR`) tanto en la **fuente de luz** como en el **material** del cubo, además de ajustar el parámetro de **brillo/exponente especular (`GL_SHININESS`)**.

Cambios necesarios en la iluminación

Solo debes actualizar la función `setup_lighting()` y modificar ligeramente el material del cubo en `draw_cube()`:

```
import pygame
from pygame.locals import *
from OpenGL.GL import *
from OpenGL.GLU import *

# (Vértices, caras y normales se mantienen igual que en el código anterior)
vertices = (
    ( 1, -1, -1), ( 1,  1, -1), (-1,  1, -1), (-1, -1, -1),
    ( 1, -1,  1), ( 1,  1,  1), (-1, -1,  1), (-1,  1,  1)
)
faces = (
    (0, 1, 2, 3), (3, 2, 7, 6), (6, 7, 5, 4),
    (4, 5, 1, 0), (1, 5, 7, 2), (4, 0, 3, 6)
)
normals = (
    (0, 0, -1), (-1, 0, 0), (0, 0, 1),
    (1, 0, 0),  (0, 1, 0),  (0, -1, 0)
)

def setup_lighting():
    glEnable(GL_LIGHTING)
    glEnable(GL_LIGHT0)
    glEnable(GL_COLOR_MATERIAL)

    # 1. Configurar los componentes de la Luz GL_LIGHT0
    ambient_light  = [0.2, 0.2, 0.2, 1.0]
    diffuse_light  = [0.7, 0.7, 0.7, 1.0]
    specular_light = [1.0, 1.0, 1.0, 1.0]  # Luz especular blanca intensa
    light_position = [3.0, 4.0, 3.0, 1.0]

    glLightfv(GL_LIGHT0, GL_AMBIENT, ambient_light)
    glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuse_light)
    glLightfv(GL_LIGHT0, GL_SPECULAR, specular_light) # <--- Activar brillo especular en la luz
    glLightfv(GL_LIGHT0, GL_POSITION, light_position)

    # 2. Configurar la respuesta especular del Material del Cubo
    mat_specular = [1.0, 1.0, 1.0, 1.0]  # El cubo reflejará brillo blanco
    shininess    = [50.0]                 # Tamaño del brillo (0-128: valores más altos = brillo más concentra

    glMaterialfv(GL_FRONT, GL_SPECULAR, mat_specular)
    glMaterialfv(GL_FRONT, GL_SHININESS, shininess)

    # glColorMaterial gestiona solo la luz ambiental y difusa a través de glColor()
    glColorMaterial(GL_FRONT, GL_AMBIENT_AND_DIFFUSE)

def draw_cube():
    glBegin(GL_QUADS)
    for i, face in enumerate(faces):
        glNormal3fv(normals[i])
        glColor3f(0.1, 0.4, 0.8)  # Azul plástico
        for vertex in face:
            glVertex3fv(vertices[vertex])
    glEnd()

def main():
    pygame.init()
    display = (800, 600)
    pygame.display.set_mode(display, DOUBLEBUF | OPENGL)
    pygame.display.set_caption("Cubo con Brillo Especular - PyOpenGL")

    gluPerspective(45, (display[0] / display[1]), 0.1, 50.0)
    glTranslatef(0.0, 0.0, -5.0)

    glEnable(GL_DEPTH_TEST)
    setup_lighting()

    rot_x, rot_y = 25, 45
    mouse_down = False
    last_mouse = (0, 0)
    clock = pygame.time.Clock()

    running = True
```

```
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT or (event.type == KEYDOWN and event.key == K_ESCAPE):
                running = False
            elif event.type == MOUSEBUTTONDOWN and event.button == 1:
                mouse_down = True
                last_mouse = event.pos
            elif event.type == MOUSEBUTTONUP and event.button == 1:
                mouse_down = False
            elif event.type == MOUSEMOTION and mouse_down:
                dx = event.pos[0] - last_mouse[0]
                dy = event.pos[1] - last_mouse[1]
                rot_x += dy * 0.5
                rot_y += dx * 0.5
                last_mouse = event.pos

        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

        glPushMatrix()
        glRotatef(rot_x, 1, 0, 0)
        glRotatef(rot_y, 0, 1, 0)

        draw_cube()

        glPopMatrix()

        pygame.display.flip()
        clock.tick(60)

    pygame.quit()

if __name__ == "__main__":
    main()
```

Explicación de las nuevas propiedades

Propiedad	Configuración	Efecto visual
`GL_SPECULAR` (Luz)	`[1.0, 1.0, 1.0, 1.0]`	Define el color del rayo brillante que proyecta la fuente de luz sobre los objetos.
`GL_SPECULAR` (Material)	`glMaterialfv(...)`	Establece el color de rebote especular del objeto. Un valor blanco `(1,1,1)` simula **plástico**, mientras que usar el mismo color del objeto simula un acabado **metálico**.
`GL_SHININESS`	`[50.0]` *(Rango: 0.0 a 128.0)*	Controla la concentración del brillo: **valores bajos** (ej. 5.0) dan destellos amplios y suaves (superficie opaca); **valores altos** (ej. 100.0) crean un punto de luz diminuto y enfocado (superficie superpulida).

User prompt: ¿Cómo puedo hacer que la luz rote en círculo alrededor del cubo en PyOpenGL?

Response: Para lograr que la luz rote en un círculo alrededor del cubo, debes calcular las coordenadas de la luz en cada fotograma utilizando funciones trigonométricas (**seno** y **coseno**) aplicadas al tiempo o a un ángulo que se incremente continuamente, y luego actualizar la posición con `glLightfv(GL_LIGHT0, GL_POSITION, ...)`.

Además, para apreciar visualmente dónde está la fuente de luz en cada momento, es muy útil **dibujar una pequeña esfera blanca u objeto flotante** en la posición exacta de la luz.

Código completo con luz en rotación

Este código hace rotar la luz en el plano $X - Z$ alrededor del cubo y dibuja una esfera que representa la bombilla/fuente de luz:

```
import math
import pygame
from pygame.locals import *
from OpenGL.GL import *
from OpenGL.GLU import *

# Geometría y normales del cubo
vertices = (
    ( 1, -1, -1), ( 1,  1, -1), (-1,  1, -1), (-1, -1, -1),
    ( 1, -1,  1), ( 1,  1,  1), (-1, -1,  1), (-1,  1,  1)
```

```
    )
    faces = (
        (0, 1, 2, 3), (3, 2, 7, 6), (6, 7, 5, 4),
        (4, 5, 1, 0), (1, 5, 7, 2), (4, 0, 3, 6)
    )
    normals = (
        (0, 0, -1), (-1, 0, 0), (0, 0, 1),
        (1, 0, 0),  (0, 1, 0),  (0, -1, 0)
    )

    def init_lighting():
        glEnable(GL_LIGHTING)
        glEnable(GL_LIGHT0)
        glEnable(GL_COLOR_MATERIAL)

        ambient_light  = [0.15, 0.15, 0.15, 1.0]
        diffuse_light  = [0.9, 0.9, 0.9, 1.0]
        specular_light = [1.0, 1.0, 1.0, 1.0]

        glLightfv(GL_LIGHT0, GL_AMBIENT, ambient_light)
        glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuse_light)
        glLightfv(GL_LIGHT0, GL_SPECULAR, specular_light)

        # Propiedades del material del cubo
        glMaterialfv(GL_FRONT, GL_SPECULAR, [1.0, 1.0, 1.0, 1.0])
        glMaterialfv(GL_FRONT, GL_SHININESS, [60.0])
        glColorMaterial(GL_FRONT, GL_AMBIENT_AND_DIFFUSE)

    def draw_cube():
        glBegin(GL_QUADS)
        for i, face in enumerate(faces):
            glNormal3fv(normals[i])
            glColor3f(0.2, 0.5, 0.8) # Azul
            for vertex in face:
                glVertex3fv(vertices[vertex])
        glEnd()

    def draw_light_bulb(position):
        """ Dibuja una esfera pequeña sin iluminación propia para representar la bombilla """
        glDisable(GL_LIGHTING) # Desactivar iluminación para que la esfera sea auto-brillante (blanca pura)
        glPushMatrix()
        glTranslatef(position[0], position[1], position[2])
        glColor3f(1.0, 1.0, 1.0)

        # Crear una esfera pequeña con gluNewQuadric
        quad = gluNewQuadric()
        gluSphere(quad, 0.15, 16, 16)

        glPopMatrix()
        glEnable(GL_LIGHTING) # Reactivar la iluminación para el resto de la escena

    def main():
        pygame.init()
        display = (800, 600)
        pygame.display.set_mode(display, DOUBLEBUF | OPENGL)
        pygame.display.set_caption("Luz Rotatoria alrededor del Cubo - PyOpenGL")

        gluPerspective(45, (display[0] / display[1]), 0.1, 50.0)
        glTranslatef(0.0, 0.0, -7.0)

        glEnable(GL_DEPTH_TEST)
        init_lighting()

        angle = 0.0
        radius = 3.5  # Distancia de la luz al centro del cubo
        light_y = 1.5 # Altura constante de la luz

        rot_x, rot_y = 20, 30
        mouse_down = False
        last_mouse = (0, 0)
        clock = pygame.time.Clock()

        running = True
        while running:
```

```
        for event in pygame.event.get():
            if event.type == pygame.QUIT or (event.type == KEYDOWN and event.key == K_ESCAPE):
                running = False
            elif event.type == MOUSEBUTTONDOWN and event.button == 1:
                mouse_down = True
                last_mouse = event.pos
            elif event.type == MOUSEBUTTONUP and event.button == 1:
                mouse_down = False
            elif event.type == MOUSEMOTION and mouse_down:
                dx = event.pos[0] - last_mouse[0]
                dy = event.pos[1] - last_mouse[1]
                rot_x += dy * 0.5
                rot_y += dx * 0.5
                last_mouse = event.pos

        # 1. Incrementar el ángulo de la luz continuamente
        angle += 0.03

        # 2. Calcular coordenadas cartesianas (X, Z) del movimiento circular
        light_x = math.cos(angle) * radius
        light_z = math.sin(angle) * radius
        light_pos = [light_x, light_y, light_z, 1.0]

        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

        glPushMatrix()

        # Rotación del punto de vista con el ratón
        glRotatef(rot_x, 1, 0, 0)
        glRotatef(rot_y, 0, 1, 0)

        # 3. Actualizar la posición de la luz en la escena
        glLightfv(GL_LIGHT0, GL_POSITION, light_pos)

        # 4. Dibujar la bombilla (fuente de luz) y el cubo
        draw_light_bulb(light_pos)
        draw_cube()

        glPopMatrix()

        pygame.display.flip()
        clock.tick(60)

    pygame.quit()

if __name__ == "__main__":
    main()
```

Detalles técnicos clave

1. **Ecuación del círculo:**

$$\text{light_x} = \cos(\text{ángulo}) \times \text{radio}$$

$$\text{light_z} = \sin(\text{ángulo}) \times \text{radio}$$

   Al ir variando `angle` frame a frame, la luz dibuja una trayectoria circular perfeita en el plano horizontal.

2. **Ubicación del comando `glLightfv`:** Se llama **dentro de la matriz de transformación principal** (`glPushMatrix()` / `glRotatef`), lo que permite que si rotas la cámara con el ratón, la posición de la luz se transforme de forma coherente con el cubo.

3. **Visualización de la fuente (`draw_light_bulb`):** Usamos `glDisable(GL_LIGHTING)` justo antes de dibujar la esfera blanca. Esto es necesario para que la bombilla se dibuje en blanco puro uniforme y no se sombee a sí misma.

User prompt: ¿Cómo se añaden dos o más fuentes de luz independientes en PyOpenGL?

Response: Para añadir múltiples fuentes de luz en PyOpenGL, el API de OpenGL clásico soporta hasta **8 luces independientes** activadas mediante las constantes `GL_LIGHT0`, `GL_LIGHT1`, `GL_LIGHT2`, ..., `GL_LIGHT7`.

Cada luz se configura de manera completamente individual con sus propios parámetros de color (ambiental, difuso y especular) y su propia posición en el espacio 3D.

Ejemplo: Dos luces de colores rotando en direcciones opuestas

En este ejemplo configuramos:

1. **`GL_LIGHT0` (Luz Azul):** Rota en sentido horario.
2. **`GL_LIGHT1` (Luz Roja):** Rota en sentido antihorario a diferente altura.

```
import math
import pygame
from pygame.locals import *
from OpenGL.GL import *
from OpenGL.GLU import *

# Geometría y normales del cubo
vertices = (
    ( 1, -1, -1), ( 1,  1, -1), (-1,  1, -1), (-1, -1, -1),
    ( 1, -1,  1), ( 1,  1,  1), (-1, -1,  1), (-1,  1,  1)
)
faces = (
    (0, 1, 2, 3), (3, 2, 7, 6), (6, 7, 5, 4),
    (4, 5, 1, 0), (1, 5, 7, 2), (4, 0, 3, 6)
)
normals = (
    (0, 0, -1), (-1, 0, 0), (0, 0, 1),
    (1, 0, 0),  (0, 1, 0),  (0, -1, 0)
)

def init_lighting():
    glEnable(GL_LIGHTING)

    # ------------------------------------------------------------
    # 1. Configurar LUZ 0 (Azul)
    # ------------------------------------------------------------
    glEnable(GL_LIGHT0)
    glLightfv(GL_LIGHT0, GL_AMBIENT,  [0.05, 0.05, 0.1, 1.0])
    glLightfv(GL_LIGHT0, GL_DIFFUSE,  [0.1, 0.4, 1.0, 1.0])  # Azul brillante
    glLightfv(GL_LIGHT0, GL_SPECULAR, [0.2, 0.6, 1.0, 1.0])

    # ------------------------------------------------------------
    # 2. Configurar LUZ 1 (Roja)
    # ------------------------------------------------------------
    glEnable(GL_LIGHT1)
    glLightfv(GL_LIGHT1, GL_AMBIENT,  [0.1, 0.05, 0.05, 1.0])
    glLightfv(GL_LIGHT1, GL_DIFFUSE,  [1.0, 0.2, 0.2, 1.0])  # Rojo intenso
    glLightfv(GL_LIGHT1, GL_SPECULAR, [1.0, 0.5, 0.5, 1.0])

    # Configuración del material del cubo (Gris neutro para reflejar ambos colores)
    glEnable(GL_COLOR_MATERIAL)
    glMaterialfv(GL_FRONT, GL_SPECULAR, [1.0, 1.0, 1.0, 1.0])
    glMaterialfv(GL_FRONT, GL_SHININESS, [50.0])
    glColorMaterial(GL_FRONT, GL_AMBIENT_AND_DIFFUSE)

def draw_cube():
    glBegin(GL_QUADS)
    for i, face in enumerate(faces):
        glNormal3fv(normals[i])
        glColor3f(0.8, 0.8, 0.8)  # Blanco/Gris neutro
        for vertex in face:
            glVertex3fv(vertices[vertex])
    glEnd()

def draw_light_marker(position, color):
    """ Dibuja una esfera indicadora del color de la luz """
    glDisable(GL_LIGHTING)
    glPushMatrix()
    glTranslatef(position[0], position[1], position[2])
    glColor3fv(color)

    quad = gluNewQuadric()
    gluSphere(quad, 0.12, 16, 16)

    glPopMatrix()
    glEnable(GL_LIGHTING)

def main():
```

```
    pygame.init()
    display = (800, 600)
    pygame.display.set_mode(display, DOUBLEBUF | OPENGL)
    pygame.display.set_caption("Múltiples Fuentes de Luz - PyOpenGL")

    gluPerspective(45, (display[0] / display[1]), 0.1, 50.0)
    glTranslatef(0.0, 0.0, -7.0)

    glEnable(GL_DEPTH_TEST)
    init_lighting()

    angle = 0.0
    rot_x, rot_y = 20, 30
    mouse_down = False
    last_mouse = (0, 0)
    clock = pygame.time.Clock()

    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT or (event.type == KEYDOWN and event.key == K_ESCAPE):
                running = False
            elif event.type == MOUSEBUTTONDOWN and event.button == 1:
                mouse_down = True
                last_mouse = event.pos
            elif event.type == MOUSEBUTTONUP and event.button == 1:
                mouse_down = False
            elif event.type == MOUSEMOTION and mouse_down:
                dx = event.pos[0] - last_mouse[0]
                dy = event.pos[1] - last_mouse[1]
                rot_x += dy * 0.5
                rot_y += dx * 0.5
                last_mouse = event.pos

        angle += 0.02

        # Posición Luz 0 (Azul) -> Rota en sentido horario a Y = 1.0
        pos_light0 = [math.cos(angle) * 3.5, 1.0, math.sin(angle) * 3.5, 1.0]

        # Posición Luz 1 (Roja) -> Rota en sentido antihorario a Y = -1.0
        pos_light1 = [math.sin(angle) * 3.5, -1.0, math.cos(angle) * 3.5, 1.0]

        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

        glPushMatrix()
        glRotatef(rot_x, 1, 0, 0)
        glRotatef(rot_y, 0, 1, 0)

        # Actualizar las posiciones de ambas fuentes de luz
        glLightfv(GL_LIGHT0, GL_POSITION, pos_light0)
        glLightfv(GL_LIGHT1, GL_POSITION, pos_light1)

        # Dibujar marcadores visuales para cada bombilla
        draw_light_marker(pos_light0, (0.2, 0.5, 1.0)) # Marcador Azul
        draw_light_marker(pos_light1, (1.0, 0.2, 0.2)) # Marcador Rojo

        # Dibujar el cubo
        draw_cube()

        glPopMatrix()

        pygame.display.flip()
        clock.tick(60)

    pygame.quit()

if __name__ == "__main__":
    main()
```

Puntos clave para gestionar múltiples luces

1. **Habilitación independiente:** Debes ejecutar `glEnable(GL_LIGHT0)`, `glEnable(GL_LIGHT1)`, etc., para cada luz que desees activar. Si no llamas a `glEnable` para una luz en particular, OpenGL ignorará su configuración.

2. **Suma de iluminación:** OpenGL acumula los efectos de iluminación. El color final de un píxel es la suma del aporte ambiental, difuso y especular de **todas** las luces activas simultáneamente.
3. **Direccional vs. Posicional:** El cuarto componente del array de posición `[X, Y, Z, W]` define el tipo de luz:
    - **`W = 1.0` (Posicional):** La luz emana de un punto fijo específico en el espacio (como una bombilla o linterna).
    - **`W = 0.0` (Direccional):** La luz se considera infinita e incide paralela en todos los objetos (como la luz del sol), y el vector `[X, Y, Z]` representa la *dirección* de los rayos.

User prompt: ¿Cómo puedo cargar y renderizar un archivo de modelo 3D .OBJ en PyOpenGL en lugar de un cubo primitivo?

Response: Para cargar y renderizar un modelo 3D en formato `.OBJ` en PyOpenGL existen dos alternativas principales:

1. **Usar una librería especializada como `pywavefront`** (La opción más rápida y recomendada para proyectos reales).
2. **Crear tu propio cargador/parser desde cero** (Ideal para aprender la estructura interna del formato `.OBJ`).

A continuación se detalla la primera opción usando `pywavefront`, ya que gestiona automáticamente la lectura de vértices, caras, normales, colores y materiales (`.mtl`).

Paso 1: Instalar dependencias

Asegúrate de instalar `pywavefront` junto con Pygame y PyOpenGL:

```
pip install PyOpenGL PyOpenGL_accelerate pygame pywavefront
```

Paso 2: Código completo para cargar y renderizar el `.OBJ`

Guarda tu archivo `.obj` (por ejemplo, `modelo.obj`) en el mismo directorio de tu script de Python.

```
import sys
import pygame
from pygame.locals import *
from OpenGL.GL import *
from OpenGL.GLU import *
import pywavefront
from pywavefront import visualization

def init_lighting():
    glEnable(GL_LIGHTING)
    glEnable(GL_LIGHT0)
    glEnable(GL_COLOR_MATERIAL)

    # Configuración de luz para resaltar el modelo 3D
    glLightfv(GL_LIGHT0, GL_AMBIENT,  [0.2, 0.2, 0.2, 1.0])
    glLightfv(GL_LIGHT0, GL_DIFFUSE,  [0.8, 0.8, 0.8, 1.0])
    glLightfv(GL_LIGHT0, GL_SPECULAR, [1.0, 1.0, 1.0, 1.0])
    glLightfv(GL_LIGHT0, GL_POSITION, [5.0, 5.0, 5.0, 1.0])

    glColorMaterial(GL_FRONT_AND_BACK, GL_AMBIENT_AND_DIFFUSE)

def load_obj_model(filename):
    """
    Carga el modelo .OBJ optimizando el cálculo de normales y caras.
    """
    try:
        scene = pywavefront.Wavefront(
            filename,
            collect_faces=True,
            create_materials=True
        )
        return scene
    except Exception as e:
        print(f"Error al cargar el archivo .OBJ: {e}")
        sys.exit()

def main():
    pygame.init()
    display = (800, 600)
    pygame.display.set_mode(display, DOUBLEBUF | OPENGL)
    pygame.display.set_caption("Cargador de Modelos .OBJ - PyOpenGL")
```

```
    # Configuración de la cámara / proyección
    gluPerspective(45, (display[0] / display[1]), 0.1, 100.0)

    # Mover la cámara hacia atrás para visualizar el modelo
    # Nota: Si el modelo es muy grande o muy pequeño, ajusta la distancia Z o usa glScale()
    glTranslatef(0.0, -1.0, -10.0)

    glEnable(GL_DEPTH_TEST)
    init_lighting()

    # Cargar el archivo .OBJ (cambia 'modelo.obj' por el nombre de tu archivo)
    scene = load_obj_model('modelo.obj')

    rot_x, rot_y = 0, 0
    mouse_down = False
    last_mouse = (0, 0)
    clock = pygame.time.Clock()

    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT or (event.type == KEYDOWN and event.key == K_ESCAPE):
                running = False
            elif event.type == MOUSEBUTTONDOWN and event.button == 1:
                mouse_down = True
                last_mouse = event.pos
            elif event.type == MOUSEBUTTONUP and event.button == 1:
                mouse_down = False
            elif event.type == MOUSEMOTION and mouse_down:
                dx = event.pos[0] - last_mouse[0]
                dy = event.pos[1] - last_mouse[1]
                rot_x += dy * 0.5
                rot_y += dx * 0.5
                last_mouse = event.pos

        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

        glPushMatrix()

        # Aplicar la rotación interactiva con el ratón
        glRotatef(rot_x, 1, 0, 0)
        glRotatef(rot_y, 0, 1, 0)

        # Asignar un color por defecto si el .OBJ no define materiales
        glColor3f(0.7, 0.7, 0.7)

        # Renderizar automáticamente las caras, normales y geometrías cargadas por pywavefront
        visualization.draw(scene)

        glPopMatrix()

        pygame.display.flip()
        clock.tick(60)

    pygame.quit()

if __name__ == "__main__":
    main()
```

Consideraciones prácticas al trabajar con modelos `.OBJ`

1. **Escala del modelo:** Muchos modelos exportados de Blender o software 3D pueden ser extremadamente grandes o diminutos. Si ejecutas el código y la pantalla queda negra/gris, ajusta la escala insertando `glScalef(0.1, 0.1, 0.1)` antes de dibujar.
2. **Posición del centro del objeto:** Asegúrate de que en el software de modelado el objeto esté centrado en el origen `(0, 0, 0)`. De lo contrario, al aplicar la rotación con el ratón, el modelo girará en un eje desplazado.
3. **Materiales y Texturas (`.MTL`):** Si el modelo viene acompañado de un archivo `.mtl` y texturas `.png` / `.jpg`, coloca todos los archivos en la misma carpeta que el ejecutable para que `pywavefront` los reconozca y aplique automáticamente los mapas de textura.

User prompt: Recomiéndame algunos libros de OpenGL y Python

Response: Los libros dedicados **exclusivamente a la combinación exacta de OpenGL + Python** son escasos, ya que la mayoría de la literatura avanzada de OpenGL utiliza C o C++. Sin embargo, existen excelentes libros prácticos en Python con capítulos profundos de PyOpenGL, así como tratados fundamentales de OpenGL cuyas teorías y conceptos se traducen fácilmente a Python.

Aquí tienes los mejores libros recomendados organizados por enfoque:

1. Específicos de Python y Gráficos 3D (PyOpenGL)

- **Python Graphics: Using Pygame, PyOpenGL, and Kivy** (*B.O. Busa*)
  - **Enfoque:** Práctico e introductorio.
  - **Ideal si:** Estás empezando desde cero y quieres entender cómo Pygame y PyOpenGL trabajan juntos. Cubre la creación de ventanas, geometría básica, transformaciones 3D, iluminación básica y renderizado de mallas simples.
- **Computer Graphics Programming in OpenGL with C++** (*V. Scott Gordon & John Clevenger*)
  - **Enfoque:** OpenGL Moderno (Basado en Shaders / GLSL).
  - **Nota importante:** Aunque está escrito para C++, **la estructura conceptual, la sintaxis de los shaders en GLSL y la lógica de los VBO/VAO son idénticas en Python**. Es uno de los textos más claros y didácticos para dar el salto de OpenGL clásico (el que usa `glBegin`/`glEnd`) a OpenGL moderno.

2. Fundamentos de OpenGL (La "Biblia" de Gráficos 3D)

Si buscas dominar OpenGL a fondo (álgebra lineal, tubería de renderizado, shaders, iluminación avanzada, shadows y mapas de normales), estos son los textos de referencia internacional:

- **OpenGL SuperBible: Comprehensive Tutorial and Reference** (*Graham Sellers, Richard S. Wright Jr., et al.*)
  - **Enfoque:** La guía definitiva de OpenGL moderno.
  - **Por qué leerlo:** Explica a detalle todo el pipeline gráfico (Vertex Shaders, Fragment Shaders, Framebuffers, Texturizado, etc.). Aunque los ejemplos están en C++, entenderás cómo funciona la tarjeta gráfica a nivel interno, lo que te permitirá escribir código de PyOpenGL mucho más eficiente.
- **LearnOpenGL (El libro / Sitio Web)** (*Joey de Vries*)
  - **Enfoque:** Tutorial paso a paso (Disponible en formato libro PDF/físico y gratis en learnopengl.com).
  - **Por qué leerlo:** Es considerado por la comunidad como el **mejor recurso de aprendizaje existente**. Cubre desde abrir una ventana hasta PBR (Physically Based Rendering). Existen repositorios públicos en GitHub (como *PyOpenGLExamples*) donde la comunidad ha traducido exactamente cada capítulo de este libro a código Python.

3. Libros Complementarios Recomendados

- **Mathematics for 3D Game Programming and Computer Graphics** (*Eric Lengyel*)
  - **Enfoque:** Matemáticas aplicadas (Vectores, Matrices, Quaterniones, Transformaciones).
  - **Por qué leerlo:** Te dará la base matemática necesaria para programar cámaras 3D, cálculos de proyección y físicas sin depender ciegamente de funciones auxiliares como `gluPerspective`.

¿Por dónde empezar?

1. **Si eres principiante absoluto en 3D:** Empieza con **Python Graphics** (B.O. Busa) para afianzar conceptos en Python puro.
2. **Si buscas dominar el desarrollo 3D moderno:** Estudia **LearnOpenGL** (de Joey de Vries). Conforme leas los conceptos de GLSL y VBOs, puedes replicarlos directamente usando la biblioteca `PyOpenGL` o motores basados en PyOpenGL como **ModernGL** (una capa en Python mucho más moderna y pitónica para OpenGL 3.3+).

User prompt: ¿Qué es la biblioteca ModernGL en Python y cuáles son sus ventajas frente a PyOpenGL?

Response: **ModernGL** es una biblioteca de Python de alto nivel diseñada como una alternativa moderna a PyOpenGL. A diferencia de PyOpenGL (que es simplemente una envoltura directa sobre la API en C de OpenGL clásica), ModernGL rediseña la interacción con la GPU para adaptarla a la sintaxis y filosofía nativa de Python.

Comparativa: PyOpenGL vs. ModernGL

Característica	PyOpenGL	ModernGL
Estilo de API	Código estilo C, explícito y verboso.	Objeto-Orientado, limpio y sintaxis *pythonica*.
Versión OpenGL	Basado en el pipeline fijo clásico (`glBegin`/`glEnd`, `glLight`).	Obliga a usar **OpenGL 3.3+ Moderno** (Shaders GLSL, VBOs, VAOs).

Característica	PyOpenGL	ModernGL
Gestión de Memoria	Manual y propensa a *memory leaks* o punteros nulos.	Automática con contexto administrado y objetos de Python.
Curva de aprendizaje	Alta complejidad inicial para crear un buffer básico.	Muy rápida para cargar Shaders y dibujar geometrías.
Rendimiento	Regular (alto costo por llamadas repetidas desde Python).	**Alto** (los bucles intensivos ocurren internamente en C).

Principales ventajas de ModernGL

- **Simplificación drástica de Shaders y Buffers:** En lugar de llamar a 10 funciones distintas para compilar un Shader, crear un VBO y vincular un VAO en PyOpenGL, en ModernGL se logra en 3 líneas de código orientadas a objetos.
- **Sin código deprecado:** Elimina por completo las funciones obsoletas de OpenGL clásico (Pipeline Fijo), forzándote a programar con las mejores prácticas actuales de la industria (Pipeline Programable).
- **Integración con bibliotecas científicas:** Permite transferir datos de memoria directamente hacia y desde **NumPy**, **Pillow** o **PyTorch** de forma ultrarrápida (ideal para simulación científica o renderizado *offscreen*).
- **Renderizado sin ventana (Headless):** Permite procesar gráficos 3D, generar imágenes o procesar shaders en servidores o entornos sin interfaz gráfica sin necesidad de inicializar sistemas de ventanas como Pygame o GLFW.

Ejemplo de código: Crear un Triángulo en ModernGL

```
import moderngl
import numpy as np

# 1. Crear el contexto de ModernGL (por ejemplo, mediante moderngl-window)
ctx = moderngl.create_context(standalone=True)

# 2. Compilar Shaders (Vertex + Fragment) en una sola llamada
prog = ctx.program(
    vertex_shader='''
        #version 330
        in vec2 in_vert;
        void main() {
            gl_Position = vec4(in_vert, 0.0, 1.0);
        }
    ''',
    fragment_shader='''
        #version 330
        out vec4 fragColor;
        void main() {
            fragColor = vec4(1.0, 0.0, 0.0, 1.0);
        }
    '''
)

# 3. Crear VBO con NumPy y configurar el VAO automáticamente
vertices = np.array([-0.6, -0.6,  0.6, -0.6,  0.0, 0.6], dtype='f4')
vbo = ctx.buffer(vertices)
vao = ctx.simple_vertex_array(prog, vbo, 'in_vert')

# 4. Dibujar
# vao.render(moderngl.TRIANGLES)
```

ModernGL es la opción ideal si vas a iniciar un proyecto nuevo desde cero y buscas escribir código limpio sin pelear con la complejidad verbosa del lenguaje C.

User prompt: ¿Dónde puedo encontrar la traducción de los tutoriales de LearnOpenGL a código en Python?

Response: Existen varios repositorios comunitarios en GitHub que han traducido paso a paso los tutoriales de **LearnOpenGL** (escritos originalmente en C++) a **Python**.

Los mejores repositorios disponibles son:

1. Traducido a PyOpenGL (PyGame / GLFW)

- **PyOpenGLExamples (por Todraffe)**

- **Qué incluye:** Traducción fiel de la gran mayoría de las lecciones de LearnOpenGL (Creación de ventana, Shaders, Texturas, Iluminación Phong/Blinn-Phong, Carga de Modelos, Shadow Maps, etc.).
- **Librerías usadas:** `PyOpenGL`, `GLFW` (o `pygame`) y `PyGLM` (para sustituir las matemáticas de `glm` en C++ por objetos de Python).

- **LearnOpenGL-Python (por Warchoke)**
  - **Qué incluye:** La sección de *Getting Started* e *Lighting* reescrita en Python utilizando **ModernGL** y **PyOpenGL**.
  - **Ideal para:** Ver la diferencia entre implementar el pipeline con PyOpenGL o con bibliotecas modernas de Python.

2. Traducido a ModernGL

Si prefieres usar **ModernGL** en lugar del PyOpenGL tradicional (lo cual simplifica bastante el código al evitar las llamadas verbosas de C), estos repositorios son las mejores referencias:

- **moderngl-tutorials (por Pyglet / ModernGL Community)**
  - Contiene reimplementaciones de los conceptos de LearnOpenGL (sistemas de cámaras 3D, iluminaciones múltiples, postprocesamiento) adaptados a las mejores prácticas de Python.

Mapeo de librerías (C++ → Python)

Cuando estudies el libro o la web de LearnOpenGL con estos repositorios, verás que las bibliotecas de C++ se corresponden con las siguientes en Python:

Biblioteca original en C++ (LearnOpenGL)	Equivalente en Python
GLFW / GLUT	`glfw` o `pygame`
GLAD / GLEW	Gestionado automáticamente por `PyOpenGL` o `moderngl`
GLM (Matemáticas 3D: Vectores/Matrices)	`pyglm` (réplica exacta de GLM) o `numpy`
stb_image.h (Carga de imágenes)	`Pillow` (`PIL`)
Assimp (Carga de modelos 3D)	`pyassimp` o `pywavefront`

Consejo de estudio

Abre el tutorial original en **learnopengl.com** para leer las explicaciones teóricas y matemáticas (son idénticas para cualquier lenguaje) y mantén al lado el archivo `.py` equivalente del repositorio **PyOpenGLExamples** para ver cómo se escribe la sintaxis en Python.
